

## FLAT Unit-5 Undecidability

Topics to cover:

- \*1 Undecidable problems and Computable function ✓
- \*1 PCP ✓
- \*1 MPCP ✓
- \*1 Recursive and Recursively enumerable languages, properties ✓
- \*1 Universal TM ✓
- \*1 Tractable and Intractable problems
- \*1 P and NP completeness ✓
- \*1 3 CNF SAT problems

### Undecidable problem

It is a problem that has high chance of not only searching to yes or no in algorithm, they may stuck in the machine and never be moved

Ex: The halting problem: Some computations that halts for decision no, remain halted

### Computable function

The algorithm that will not lead the computation to undecidability and reaches the output is called computable function

## The Post correspondence Problem / PCP

It is one of the undecidable problems involving "dominoes" where top and bottom strings. A solution exists if top string exactly matches the bottom string. Proving undecidability.

Approach: Take problem that is already proven to be undecidable. If both are equivalent, then this is not decidable.

Ex:  $\Sigma \{0, 1\}$

$A \Rightarrow w_1, w_2, \dots, w_n$

$B \Rightarrow x_1, x_2, \dots, x_k$

RP should have solution at any combination

| p | list A | list B |
|---|--------|--------|
|   | $w_i$  | $x_i$  |
| 1 | 1      | 111    |
| 2 | 10111  | 10     |
| 3 | 10     | 0      |

(2, 1, 1, 3) can be used

$w^0 = 101111110$   
 $x^0 = 101111110$

These are matched depending on combination.  
 $\therefore$  PCP are undecidable



(2)

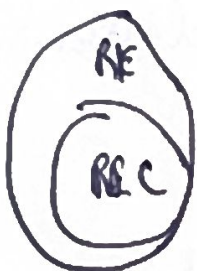
MPCP (modified Post correspondence problem)

\* It is restricted ver of PCP also proves undecidability

## Recursive and Recursively enumerable

Recursive Enumerable:

- \* It is a language accepted by TM
- \* It Halts & rejects but also has a chance of never halt and infinite loop for rejected strings, that's why we use REC (recursive languages) that never is loop



Properties:

Union  
Intersection  
Concatenation  
Kleene closure  
Complement

P, NP } NP - Hard problem  
 ↓  
 NP completeness

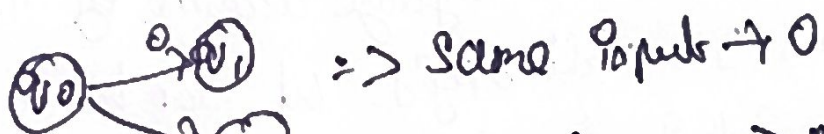
### Non-Deterministic Algorithm (NP):

Cannot be solved in polynomial time

Can be verified in polynomial time

\* Output cannot be predicted properly even when we know input

\* Same input will give different output for round of execution.



$\Rightarrow$  Same input  $\rightarrow$  0

$\Rightarrow q_1$  and  $q_2 \Rightarrow$  non deterministic algorithm



$\Rightarrow$  deterministic algorithm

\* Algorithm will take more than one part

\* We cannot determine next step of execution

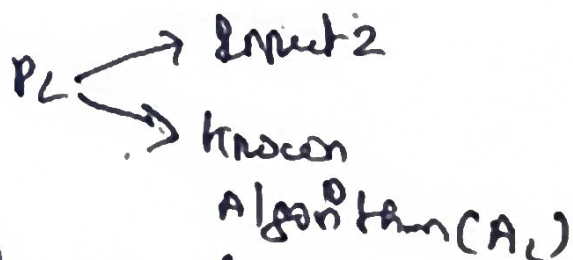
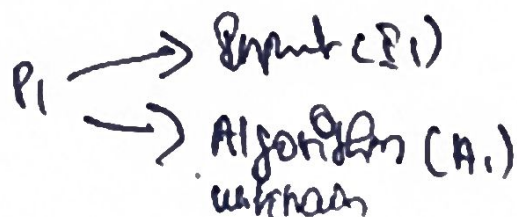
\* Only approximate solution, no accurate solution

Ex: Monte Carlo problems, Genetic algorithm

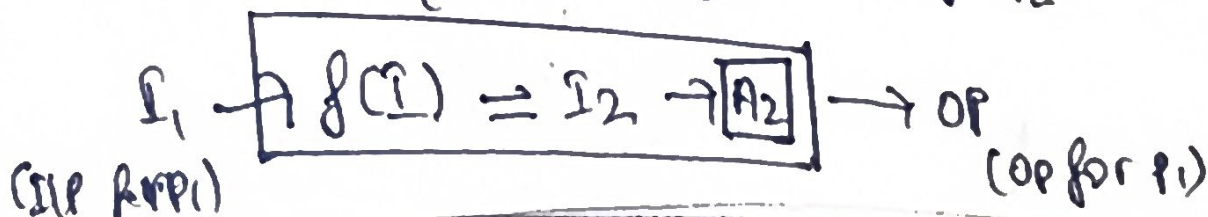
Induction:

no/yes

Consider we have two decision problems  $\rightarrow P_1, P_2$



If  $P_1$ 's input  $I_1$  can be used for  $P_2$  algorithm  $A_2$  and using  $I_1$  we can get soln of  $P_2$





P-hard

⑤

A problem is NP hard if every problem in NP can be polynomially reduced

If all these can be reduced to X, it is called NP hard

NP-complete

A problem is called NP complete if the problem is

$\Rightarrow P \subseteq \text{in } NP$

$\Rightarrow P \subseteq \text{in } NP \text{ Hard}$



NPC

Ex: Subset sum problem:

$S = \{-1, 4, 7, 10, 6, 2, 13\}$

$n = 8$

$A = \{-1, 6\}$

$B = \{2, 6\}$

Halting Problem

It is undecidable, it is not a problem, it just asks question "Is it possible to tell whether a given machine will halt for some given IP"

Input  $\Rightarrow$  A TM A I/P string  $w$   
Problem  $\Rightarrow$  Does the TM finish computation of the string  $w$  in a finite no of steps Ans: Yes/no

Proof :

Let's assume TM exists to solve this problem & then we will show it is contradicting itself.

→ Let's call this TM is a halting machine that produce "yes" or "no" in a finite amount of time

→ If the halting machine produce in a finite amount of time O/P comes as "yes" or "no"

Block diagram of halting machine:

IP String → Halting Machine → yes (H/M halts on yes)  
→ no (H/M does not halt)

→ Now construct halting machine

IP String → HM → yes (do not loop)  
→ no (halts)

### Universal Turing machine

It is a theoretical device, it is able to run any Turing Machine on its tape, an idea that represents general computation.

New Turing Mach. → UTM → returns a H or loop with ω



④  
Tractable and intractable problems

Tractable: Problems that can be solved in polynomial time by algorithm

The time required is directly proportional to polynomial function of input size

Intractable: No known polynomial-time algorithm.